Volume 1A

100-Day Python Software Engineering Challenge

Problems 1-10: Beginner to Intermediate Real-World Practice

Prepared for Nishanth M

Focus: Python Basics, Functions, OOP Thinking, and File Handling

How to Use This PDF

Solve one problem per day. Do not copy code from the internet. First understand the business scenario, then write the algorithm, then code the solution.

For each problem, start with a simple version. After it works, add bonus features. This is how real software is built: basic version first, improved version next.

After solving, test your program with valid input, wrong input, and edge cases. Save your code in GitHub with a clear README.

Table of Contents

Day 1: College Canteen Billing System - Beginner

Day 2: Student Attendance Alert System - Beginner

Day 3: Library Fine Calculator - Beginner

Day 4: ATM Simulator - Beginner

Day 5: Electricity Bill Generator - Beginner

Day 6: Personal Expense Tracker - Beginner to Intermediate

Day 7: Employee Payroll System - Beginner to Intermediate

Day 8: Inventory Management System - Intermediate

Day 9: Bus Reservation System - Intermediate

Day 10: Hospital Appointment System - Intermediate

Day 1: College Canteen Billing System

Difficulty: Beginner

Business Scenario

A college canteen currently writes bills manually. During break time, the staff struggle to calculate totals quickly, apply discounts, and track daily sales. The canteen needs a simple Python application to generate bills accurately.

Objective

Build a menu-driven billing program that accepts multiple food orders, calculates the total amount, applies discounts, prints a receipt, and optionally saves the bill details using file handling.

Functional Requirements

1. Display a fixed menu with item names and prices.

2. Allow a customer to order multiple items in one bill.

3. Ask for item number/name and quantity for each order.

4. Calculate subtotal for every item and the total bill.

5. Apply 10% discount if the total bill is greater than Rs. 200.

6. Display a clean receipt with ordered items, quantities, subtotal, discount, and final amount.

7. Ask whether the next customer wants to place an order.

8. Save each bill in a text file with bill number and final amount.

Input Specifications

Customer name, selected item number/name, quantity, and whether the customer wants to add more items.

Output Specifications

Formatted bill receipt showing item-wise details, total, discount, final amount, and saved bill confirmation.

Constraints and Validation Rules

- Quantity must be a positive integer.

- Invalid item numbers must show an error message.

- The program should not crash for wrong input.

- Use only Python basics, functions, dictionaries/lists, loops, and file handling.

Sample Input

```
Customer Name: Arun
Select Item: Burger
Quantity: 2
Add more? yes
Select Item: Tea
Quantity: 3
Add more? no
```

Sample Output

```
----- College Canteen Bill -----
Customer: Arun
Burger x 2 = Rs. 160
Tea x 3 = Rs. 30
Total Amount: Rs. 190
Discount: Rs. 0
Final Amount: Rs. 190
Bill saved successfully.
```

Skills Covered	variables, conditions, loops, functions, dictionaries, file handling
Bonus Features	- Add GST calculation. - Generate unique bill numbers. - Create a daily sales report. - Find the most sold item.
Interview Discussion Questions	- How will you prevent invalid quantity input? - Why is a dictionary suitable for storing menu items? - How will you store daily bills in a file?

Day 2: Student Attendance Alert System

Difficulty: Beginner

Business Scenario

A class advisor wants to identify students who have low attendance before the exam registration process. Manual calculation is slow and error-prone, especially for a large class.

Objective

Create a Python program that calculates attendance percentage for students and marks them as eligible or not eligible for exams based on the minimum attendance requirement.

Functional Requirements

1. Accept details for multiple students.

2. Input total classes conducted and classes attended.

3. Calculate attendance percentage.

4. If attendance is below 75%, show Not Eligible for Exam.

5. If attendance is 75% or above, show Eligible for Exam.

6. Display all student results in a clean report.

7. Save shortage students separately in a text file.

Input Specifications

Student name, roll number, total classes conducted, and classes attended.

Output Specifications

Attendance percentage and exam eligibility status for each student.

Constraints and Validation Rules

- Classes attended cannot be greater than total classes.

- Total classes must be greater than zero.

- Attendance percentage should be rounded to two decimals.

Sample Input

```
Student Name: Nishanth
Roll No: 25CS170
Total Classes: 100
Classes Attended: 72
```

Sample Output

```
----- Attendance Report -----
Name: Nishanth
Roll No: 25CS170
Attendance: 72.00%
Status: Not Eligible for Exam
```

Skills Covered	input validation, percentage calculation, conditions, loops, file handling
Bonus Features	- Process the whole class using a list of students. - Show students near shortage between 70% and 75%. - Generate a department-wise report.

Interview Discussion Questions	- How will you handle total classes as zero? - Why should classes attended not exceed total classes? - How can this be converted into an OOP program?

Day 3: Library Fine Calculator

Difficulty: Beginner

Business Scenario

A college library allows students to borrow books for a fixed number of days. If students return books late, the librarian calculates the fine manually. The library wants a simple fine calculator.

Objective

Develop a Python application that calculates library fines based on due date, return date, and fine rules.

Functional Requirements

1. Accept student name, book name, due day, and return day.

2. Calculate number of overdue days.

3. Apply fine rules based on delay.

4. Display fine amount and return status.

5. Allow calculation for multiple students.

6. Save fine records in a text file.

Input Specifications

Student name, book title, due date/day number, return date/day number.

Output Specifications

Overdue days, fine amount, and status message.

Constraints and Validation Rules

- Return day cannot be negative.

- If return day is before or equal to due day, fine is zero.

- Use simple integer day values at beginner level.

Sample Input

```
Student: Meena
Book: Python Basics
Due Day: 10
Return Day: 15
```

Sample Output

```
----- Library Fine Receipt -----
Student: Meena
Book: Python Basics
Overdue Days: 5
Fine Amount: Rs. 25
Status: Returned with fine
```

Skills Covered	conditions, arithmetic, functions, loops, file handling
Bonus Features	- Use actual dates using datetime module. - Add book issue and return history. - Create a search feature by student name.
Interview Discussion Questions	- How will you calculate fine if the book is not late? - What data should be stored in the file? - How can you change fine rules without rewriting the full program?

Day 4: ATM Simulator

Difficulty: Beginner

Business Scenario

A bank wants a basic ATM simulation for training students. The application should allow users to check balance, deposit money, withdraw money, and see transactions.

Objective

Build a menu-driven ATM simulator that manages balance and validates transactions.

Functional Requirements

1. Start with an initial account balance.
2. Show options: check balance, deposit, withdraw, transaction history, exit.
3. Allow deposit of valid positive amounts.
4. Allow withdrawal only if sufficient balance exists.
5. Store every transaction in a list.
6. Display transaction history before exit.
7. Save transaction history to a file.

Input Specifications

Menu choice, deposit amount, withdrawal amount.

Output Specifications

Updated balance, transaction status, and transaction history.

Constraints and Validation Rules

- Deposit and withdrawal amounts must be positive.
- Withdrawal cannot exceed balance.
- Invalid menu choice should show a clear message.

Sample Input

```
Initial Balance: Rs. 5000
Choice: Withdraw
Amount: 1200
Choice: Check Balance
```

Sample Output

```
Withdrawal successful.
Current Balance: Rs. 3800
Transaction History:
- Withdraw: Rs. 1200
```

Skills Covered	menu-driven programming, loops, conditions, lists, file handling
Bonus Features	- Add PIN verification. - Limit wrong PIN attempts to 3. - Create an Account class using OOP.
Interview Discussion Questions	- Why should withdrawal check balance first? - How will you store transaction history? - What extra security is needed in a real ATM?

Day 5: Electricity Bill Generator

Difficulty: Beginner

Business Scenario

An electricity board wants to generate monthly bills for domestic customers. The amount depends on units consumed and different slab rates.

Objective

Create a Python program that calculates electricity bills using slab-based pricing.

Functional Requirements

1. Accept customer name, service number, and units consumed.

2. Calculate bill using slab rates.

3. Display customer details and bill amount.

4. Add fixed meter charge.

5. Show warning for very high usage.

6. Generate bills for multiple customers.

7. Save bill records to a file.

Input Specifications

Customer name, service number, and monthly units consumed.

Output Specifications

Detailed electricity bill with units, slab amount, fixed charge, and final amount.

Constraints and Validation Rules

- Units must not be negative.

- Slab calculation must be accurate.

- Use functions to keep bill calculation separate.

Sample Input

```
Customer: Ravi
Service No: EB1024
Units: 180
```

Sample Output

```
----- Electricity Bill -----
Customer: Ravi
Service No: EB1024
Units Consumed: 180
Energy Charge: Rs. 540
Fixed Charge: Rs. 50
Total Bill: Rs. 590
```

Skills Covered	slab logic, functions, conditions, loops, file handling
Bonus Features	- Add commercial customer category. - Generate monthly summary. - Find highest usage customer.

Interview Discussion Questions	- How will you test slab calculation? - Why should bill logic be inside a function? - How can you add new slab rates easily?

Day 6: Personal Expense Tracker

Difficulty: Beginner to Intermediate

Business Scenario

A student wants to track daily expenses such as food, travel, books, and mobile recharge. The student wants to know total spending and where most money is spent.

Objective

Build an expense tracker that records expenses, calculates totals, groups expenses by category, and saves records to a file.

Functional Requirements

1. Add expense with date, category, description, and amount.

2. Display all expenses.

3. Calculate total spending.

4. Show category-wise spending.

5. Find the highest expense.

6. Save expense records in a file.

7. Allow user to view previous records.

Input Specifications

Date, category, description, amount, and menu choice.

Output Specifications

Expense list, total amount spent, category summary, and highest expense.

Constraints and Validation Rules

- Amount must be positive.

- Category cannot be empty.

- Use file handling to store data permanently.

Sample Input

```
Date: 20-06-2026
Category: Food
Description: Lunch
Amount: 80
```

Sample Output

```
Expense added successfully.
Total Spending: Rs. 80
Highest Expense: Lunch - Rs. 80
Category Summary:
Food: Rs. 80
```

Skills Covered	lists, dictionaries, functions, file handling, basic analytics
Bonus Features	- Set monthly budget and show warning. - Export expenses to CSV. - Add search by category or date.

Interview Discussion Questions	- Why is category-wise grouping useful? - How will you store records so they can be read later? - How can this project be improved with OOP?

Day 7: Employee Payroll System

Difficulty: Beginner to Intermediate

Business Scenario

A small company calculates employee salary manually every month. The salary includes basic pay, allowance, deduction, and net salary.

Objective

Create a Python payroll system that calculates net salary and generates salary slips for employees.

Functional Requirements

1. Accept employee ID, name, basic salary, allowance, and deduction.

2. Calculate gross salary.

3. Calculate net salary.

4. Display salary slip.

5. Process multiple employees.

6. Save salary slip records in a file.

7. Show total payroll amount for the company.

Input Specifications

Employee ID, employee name, basic salary, allowance, deduction.

Output Specifications

Salary slip with gross salary, deduction, net salary, and payroll summary.

Constraints and Validation Rules

- Salary values must not be negative.

- Net salary should not be less than zero.

- Use functions for salary calculations.

Sample Input

```
Employee ID: E101
Name: Kumar
Basic Salary: 20000
Allowance: 5000
Deduction: 2000
```

Sample Output

```
----- Salary Slip -----
Employee: Kumar
Gross Salary: Rs. 25000
Deduction: Rs. 2000
Net Salary: Rs. 23000
```

Skills Covered	arithmetic, functions, loops, file handling, records
Bonus Features	- Create Employee class. - Add tax calculation. - Generate department-wise payroll report.

Interview Discussion Questions	- What is the difference between gross and net salary? - Why should salary calculations be reusable functions? - How will you handle invalid salary input?

Day 8: Inventory Management System

Difficulty: Intermediate

Business Scenario

A small shop owner wants to manage products and stock. The owner needs to add products, update stock, check low-stock items, and view inventory value.

Objective

Develop an inventory system that manages product records and stock levels using Python.

Functional Requirements

1. Add product with product ID, name, price, and quantity.

2. Display all products.

3. Update stock quantity after purchase or sale.

4. Search product by ID or name.

5. Show low-stock products below a threshold.

6. Calculate total inventory value.

7. Save product records to a file.

Input Specifications

Product ID, product name, price, quantity, and menu choice.

Output Specifications

Product list, stock update message, low-stock report, and inventory value.

Constraints and Validation Rules

- Product ID should be unique.

- Price and quantity must not be negative.

- Stock cannot become negative after sale.

Sample Input

```
Product ID: P101
Name: Notebook
Price: 50
Quantity: 20
Low Stock Limit: 5
```

Sample Output

```
Product added successfully.
Inventory Value for Notebook: Rs. 1000
Low Stock Status: No
```

Skills Covered	dictionaries/lists, search, update, file handling, OOP optional
Bonus Features	- Use Product class. - Add delete product feature. - Generate sales report. - Export inventory to CSV.
Interview Discussion Questions	- Why should product ID be unique? - How will you avoid negative stock? - What file format is best for storing inventory data?

Day 9: Bus Reservation System

Difficulty: Intermediate

Business Scenario

A private bus operator wants a simple reservation system to manage seat booking and cancellation. The system should prevent duplicate bookings.

Objective

Build a Python bus reservation system that displays seats, books seats, cancels bookings, and stores passenger details.

Functional Requirements

1. Display available and booked seats.

2. Accept passenger name, age, and seat number.

3. Book a seat only if it is available.

4. Cancel booking using seat number.

5. Show passenger list.

6. Calculate total booked seats and available seats.

7. Save booking details to a file.

Input Specifications

Passenger details, seat number, booking/cancellation choice.

Output Specifications

Seat availability chart, booking confirmation, cancellation confirmation, passenger list.

Constraints and Validation Rules

- Seat number must be within valid range.

- Already booked seat cannot be booked again.

- Only booked seats can be cancelled.

Sample Input

```
Total Seats: 10
Passenger: Priya
Age: 20
Seat No: 5
```

Sample Output

```
Seat 5 booked successfully for Priya.
Booked Seats: 1
Available Seats: 9
```

Skills Covered	lists, conditions, search, file handling, basic OOP
Bonus Features	- Add ticket fare calculation. - Generate ticket number. - Add route and timing details.
Interview Discussion Questions	- How will you represent seats in Python? - How do you prevent duplicate booking? - How can this system be extended to multiple buses?

Day 10: Hospital Appointment System

Difficulty: Intermediate

Business Scenario

A small clinic receives patient appointments manually through phone calls. Sometimes two patients are scheduled for the same doctor and time. The clinic needs a simple appointment scheduler.

Objective

Create a Python appointment system that registers patients, schedules appointments, prevents duplicate time slots, and displays daily appointments.

Functional Requirements

1. Add patient details such as name, age, phone number, and problem.

2. Allow selection of doctor name and appointment time.

3. Prevent double booking for the same doctor and time.

4. Display all appointments for the day.

5. Search appointment by patient name or phone number.

6. Cancel an appointment.

7. Save appointment records to a file.

Input Specifications

Patient details, doctor name, appointment date/time, menu choice.

Output Specifications

Appointment confirmation, daily appointment list, search results, cancellation message.

Constraints and Validation Rules

- Phone number should not be empty.

- Same doctor cannot have two appointments at same time.

- Appointment time should follow a valid format.

Sample Input

```
Patient: Suresh
Age: 45
Doctor: Dr. Kumar
Time: 10:30 AM
Problem: Fever
```

Sample Output

```
Appointment booked successfully.
Patient: Suresh
Doctor: Dr. Kumar
Time: 10:30 AM
Status: Confirmed
```

Skills Covered	lists/dictionaries, validation, search, file handling, OOP
Bonus Features	- Create Patient and Appointment classes. - Add doctor availability slots. - Generate appointment ID. - Create daily appointment report.

Interview Discussion Questions	- How will you check for duplicate appointments? - What information should be stored for each patient? - How can file handling help after closing the program?

Daily Solving Checklist

- Read the problem twice before coding.
- Write inputs, outputs, and constraints in your notebook.
- Create a simple algorithm in 5-8 steps.
- Build the basic version first.
- Test with correct input and wrong input.
- Add file handling only after the main logic works.
- Refactor using functions or classes.
- Push code to GitHub with a README.